

# Finite Volume Euler Solver for Multi-Element Airfoil Flows

Jake Yeaman, Nicolas Polimeni, Devin Altman, Alexander Feldman, Takahiro Soeda

February 21, 2023

The purpose of this project is to develop a finite-volume solver capable of performing simulations of an inviscid, compressible flow over a three-element airfoil. To address the problem three different meshes were used - a coarse mesh of approximately 2000 cells, a medium mesh of approximately 8000 cells, and a fine mesh with approximately 32000 cells. Three Riemann Solvers were generated to be used as flux functions (Roe, Rusanov and HLLE). A unit test was successfully performed to the three of them. After that, a first order finite volume solver was developed in order to simulate the compressible Euler equations. Building on the first order solver, a second order finite volume solver was also developed. For the second order solver, three possibilities for limiting were considered: none, Barth-Jespersen (BJ) and the maximum-principle (MP) region limiter LCD. Finally, several simulations were developed considering a subsonic and a transonic flow condition. Post processing was done for the first order and second order runs. Specifically, the lift and drag coefficients as well as the pressure-coefficient distribution over the airfoil were obtained and used to perform convergence and accuracy studies.

## 1 Introduction

In this project, a first and second order finite-volume simulation of an inviscid and compressible flow is develop around a three-element airfoil. This airfoil system is depicted in in Figure 1, where the main airfoil has a unit chord. For performing the mentioned simulations, three different meshes are used. A coarse mesh of approximately

2000 cells, a medium mesh of approximately 8000 cells and a fine mesh with approximately 32000 cells.

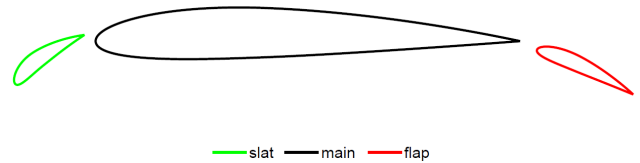


Figure 1: Three-Element Airfoil Geometry

For both solvers, three Riemann Solvers are generated and tested to be used as flux functions (Roe, Rusanov and HLLE). For the second order simulation, three possibilities for limiting are considered (none, Barth-Jespersen (BJ) and the maximum-principle (MP) region limiter LCD).

Two different flow conditions are considered, a subsonic case and a transonic case. For the subsonic case a pressure-coefficient distribution over the airfoil is generated for both the first-order and the second-order solvers.

Apart from that, convergence studies on the subsonic flow condition is developed in order to study the difference between the use of different fluxes, limiters and the order of the method. As the scalar outputs for the mentioned studies the lift and drag coefficients are used.

Moreover, on the medium and the fine meshes a transonic run is made with both the first and second order solvers. In both of them the Rusanov flux function is used and for the second order simulation the Barth-Jespersen (BJ) limiter was selected. For both cases the lift and drag coefficients are reported as well as a pressure-coefficient

distribution plot and Mach number contours.

Finally, an adaptive run is performed. Starting with the coarse mesh, the first and second order solvers are run for five adaptive iterations using the transonic flow condition. Using these results, the lift and drag coefficients and the pressure-coefficient distribution are compared with the ones obtained with the fine mesh run previously.

## 2 Methods

### 2.1 Governing Equations

For this project we consider the compressible Euler equations of gas dynamics with ratio of specific heats  $\gamma = 1.4$  as the governing equations.

$$\frac{\partial \mathbf{u}}{\partial t} + \nabla \cdot \vec{\mathbf{F}} = \mathbf{0}, \quad (1)$$

The two dimensions finite volume method will be used, so the conservative state is  $\mathbf{u} = [\rho, \rho u, \rho v, \rho E]^T$ , and  $\vec{\mathbf{F}} = \mathbf{F}\hat{x} + \mathbf{G}\hat{y}$  is the flux vector. While Eqn. 1 is unsteady, in this project we are only concerned with the steady-state solution. The free-stream state is given by

$$\vec{u}_\infty = [\rho, \rho u, \rho v, \rho E]^T$$

$$\vec{u}_\infty = [1, M_\infty \cos \alpha, M_\infty \sin \alpha, \frac{1}{(\gamma - 1)\gamma} + \frac{M_\infty^2}{2}]^T \quad (2)$$

Where  $M_\infty$  is the free-stream Mach number and  $\alpha$  is the angle of attack.

### 2.2 Flow Conditions

For this project we consider two different flow conditions. A subsonic flow and a transonic flow condition, both cases with an angle of attack of  $8^\circ$ :

- Subsonic Flow:  $M_\infty = 0.25$  and  $\alpha = 8^\circ$
- Transonic Flow:  $M_\infty = 0.5$  and  $\alpha = 8^\circ$

### 2.3 Boundary Conditions

The Three-Element Airfoil Geometry that is being simulated has 4 different boundary groups. The farfield boundary, the flap airfoil, the main airfoil and the slat airfoil. In this project consider

that the airfoil elements are solid inviscid walls, while the farfield boundary is consider a full-state boundary, where the free-stream state is specified outside the domain.

### 2.4 Flux Functions

We implement three Riemann solvers, Roe, Rusanov, and HLLE fluxes. Full state condition is used on far-field boundaries, and inviscid wall condition is used on the airfoils. In order to validate the Riemann flux functions developed, an arbitrary input is proposed, with which the flux across the edge are calculated by hand. If the hand calculations match the flux functions outputs, it is a good indication that the flux implementation is correct. This test is a unitary test and it is described in detail in the results section.

### 2.5 First-Order Finite Volume Solver

Utilizing a first-order finite volume solver to solve for the flow state of the three-element airfoil system is very important in multiple ways. First off, it offers a fairly computationally cheap method of solving for the flow state. Secondly, it can serve as an easy way to generate a fairly accurate final converged state to use as an initial guess in a higher-order method.

Here, a high level description of the implementation of the first-order solver is further outlined. The solver is dependent on various user input values. The first being a CFL number, which impacts the time-stepping calculation for the time-marching scheme. The second parameter is the desired flux function to utilize.

Each cell in the mesh is initialized to the free-stream state. Then, the solver loops through each face in the mesh and computes the desired flux by utilizing the elements adjacent to these faces. Through these fluxes, the residual can be incrementally built on each element by properly adding or subtracting the computed residual values from the adjacent cells.

The L1 residual norm is then calculated by summing the absolute value of the residual of each component of each element. If the residual norm is less than  $10^{-5}$ , the solution is considered converged.

To march forward in time and update the state on each element, the forward Euler time integration method is used. This method is outlined as shown in Eqn. 3. Specifically, this shows the time-integration method applied to each cell,  $i$ , individually.

$$u_n^{i+1} u_i^n - \frac{\Delta t}{A_i} R_i \quad (3)$$

Here, since the converged state is desired, time-stepping is done locally. Thus, meaning the time-step may carry on each cell within the mesh. The time step is computed as follows in Eqn. 4

$$\Delta t_i = \frac{2A_i \text{CFL}}{\sum_{e=1}^3 |s|_{i,e} \Delta l_{i,e}} \quad (4)$$

Here,  $s$ , represents the wave speed. This, similarly to the residual, is computed on each cell.

Finally, note that the first-order finite volume solver remains efficient in time, as it does not surpass quadratic time complexity. This is ensured by implementing a single loop, with no nested loop which scales with the mesh density.

## 2.6 Gradient Calculation and Limiting

In order to simulate the flow over the multi-element airfoil system using a second-order finite volume method, a way of interpolating the states quantities near the cell faces are necessary. Obtaining the states near the interface yield better accuracy since it offers a more accurate representation of the state at which the flux calculation is computed as opposed to computing the flux through cell average state values.

The states at the cell faces are approximated using Eqn. 5

$$\mathbf{u}(\vec{x}) = \mathbf{u}_i + \nabla \mathbf{u}|_i \cdot (\vec{x} - \vec{x}_i) \quad (5)$$

Here,  $\vec{x}$  is the location of the cell face midpoint for which an approximated state value is desired, and  $\vec{x}_i$  is the cell centroid. The approximated state relies on a state gradient term,  $\nabla \mathbf{u}|_i$ , at the cell centroid.

The gradient term is computed using the gradient theorem method. Specifically, the gradient theorem method computes the state gradient for a given cell by using its cell area, neighboring cells' centroidal state values, interface lengths, and normal vectors. Using this method, the gradient,  $\vec{L}$ , can be computed as follows in Eqn. 6

$$\vec{L} = \frac{1}{A_i} \left( \sum_{k=1}^3 \frac{\mathbf{u}_i - \mathbf{u}_k}{2} n_{ik} \Delta l_{ik} \right) \quad (6)$$

Here,  $i$  represents the cell in which the gradient is being computed on, while  $k$  represents each of the cell neighbors used to compute the gradient term. Note, that since triangular meshes are used for this project, the gradient term is calculated using up to cell neighbors.

This gradient term can be directly applied to Eqn. 5, however a limiter can be introduced to alter the gradient term. This is often used to make higher-order methods total variation diminishing (TVD), which helps avoid non-physical like oscillations in the computed states. This is achieved through slope limiting. For this project, two types of limiters were implemented for use.

The first implemented limiter was the Barth-Jespersen limiter. This is a node-based, scalar slope limiting method. This limiter depends on the estimated states at the cell nodes, denoted by the subscript  $i$ . The state estimation is computed with the current gradient term for cell  $k$ ,  $\vec{L}_k$ , shown in Eqn. 7

$$u_i^N = u_k + r_i^N \cdot \vec{L}_k \quad (7)$$

Additionally, for the Barth-Jespersen limiter,  $u_{max}$  and  $u_{min}$  are needed. These are defined as the maximum and minimum state values between cell  $k$  and its neighboring cells. With this information,

the Barth-Jespersen scaling limiter can be defined at each node as shown in Eqn. 8

$$\alpha_i^N = \begin{cases} \min \left( 1, \frac{u_{max} - u_0}{u_i^N - u_0} \right) & \text{if } u_i^N - u_0 > 0 \\ \max \left( 1, \frac{u_{min} - u_0}{u_i^N - u_0} \right) & \text{if } u_i^N - u_0 < 0 \\ 1 & \text{otherwise} \end{cases} \quad \vec{L}_k \quad (8)$$

Finally, once every scaling limiter,  $\alpha_i^N$ , is computed at each node, the final limiter,  $\alpha$ , used to scale the gradient,  $\vec{L}_k$ , is shown as follows in Eqn. 9

$$\alpha = \min \{ \alpha_i^N \} \quad (9)$$

The second type of limiter implemented is a Maximum Principle (MP) Region limiter. Specifically, a purely scaling application of this, called the Limited Central Difference (LCD) limiter, was implemented. MP limiters alter the gradient in order to constrain the solution within a region (the MP region) where a maximum principle is satisfied. Specifically, this is satisfied if the maximum value of the solution occurs at the boundary of the the cells. This MP region is constructed by defining the origin of the region at the cell center. The region is then constructed through a series of inequalities applied to each of the cell's faces. These inequalities, defined for cell 0 and its bounding faces  $j$ , are shown as follows in Eqn. 10.

$$\min(u_j - u_0, 0) \leq \vec{r}_{0j} \cdot \vec{L} \leq \max(u_j - u_0, 0) \quad (10)$$

The LCD limiting scheme is the cheapest form of MP limiting as it is a purely scaling scheme. Specifically, this scheme scales the original gradient by a scaling factor,  $\alpha$ . While the LCD scheme scales the gradient in order to lie within the MP region, this is not always the case. The scheme will snap the original gradient within the MP region if it is possible through purely, non-zero scaling. Otherwise, if gradient direction changes are needed in order to lie within the MP region, the LCD scheme cannot snap these cases within the region. The scaling factor for the LCD scheme for each edge,  $j$ , on cell 0 is defined in Eqn. 11

$$\alpha_j = \begin{cases} \frac{\max(u_j - u_0, 0)}{\vec{r}_{0j} \cdot \vec{L}} & \text{if } \vec{r}_{0j} \cdot \vec{L} > \max(u_j - u_0, 0) \\ \frac{\min(u_j - u_0, 0)}{\vec{r}_{0j} \cdot \vec{L}} & \text{if } \vec{r}_{0j} \cdot \vec{L} < \min(u_j - u_0, 0) \\ 1 & \text{otherwise} \end{cases} \quad (11)$$

Here,  $\vec{L}$  represents the gradient computed in Eqn. 6. The final scaling factor used to scale this gradient is computed as follows in Eqn. 12

$$\min(\alpha_k) \quad (12)$$

Here, we are taking the minimal value of the scaling factors computed for each face of cell 0. Specifically in the case of this project, this will be the minimum of three scaling factors due to the fact that a triangular mesh is used.

## 2.7 Second-Order Finite Volume Solver

In addition to the first-order finite volume method, a second-order method was also implemented. Specifically, the second-order method runs by first taking in the converged, first-order state solution, as the initial state guess. Unlike the first-order solver, the second-order solver computes the fluxes at the cell interfaces or boundary faces by interpolating the states near the interfaces through the use of state gradients as shown in Eqn. 5. Regarding the gradients, gradient limiting, specifically Barth-Jespersen and the LCD MP limiting techniques are added as specified by the user. Finally, instead of implementing an explicit, first-order time integration method such as Forward Euler, as used in the first-order solver, we instead needed to use a second-order, implicit method. Therefore, the RK2, multi-stage, time integration method was implemented. The general, governing equation for RK2 is shown in Eqn. 13

$$\begin{aligned} \mathbf{f}_0 &= \mathbf{f}(\mathbf{u}^n, t^n) \\ \mathbf{f}_1 &= \mathbf{f}(\mathbf{u}^n + \Delta t \mathbf{f}_0, t^{n+1}) \\ \mathbf{u}^{n+1} &= \mathbf{u}^n + \frac{1}{2} \Delta t (\mathbf{f}_0 + \mathbf{f}_1) \end{aligned} \quad (13)$$

In order to compute a time-marched state using RK2, state residual calculations need to be

computed on each cell. This residual calculation is main component of the second-order finite volume method. Here, a high-level description of the second-order finite volume implementation is outlined. First, in order to limit computational demand and runtime, the first-order finite volume method's converged state is used as the initial state guess for the second-order implementation. In order to ensure that proper memory is allocated, and thus memory does not need to be created while running the code, main data structures containing the gradient, residual, and wave speed on each cell are initialized as the proper sizes first. Furthermore, these values are initialized to be zero on each cell. Next, in order to populate the gradient term data structure on each element, each edge within the mesh is looped through. In this loop, the single neighboring cell contribution to the gradient term from Eqn. 6 is computed. This value is accordingly added or subtracted to the current face's adjacent elements depending on the direction of the face normal vector relative to the adjacent elements.

Once each face is visited and the gradient terms on each cell are populated, another loop through the mesh's faces is conducted in order to compute the residual and wave speed values on each cell. Within this loop, the flux is determined at each cell interface, using the interpolated cell interface state values on each adjacent cell using Eqn. 5. Similarly to the gradient term, these fluxes are appropriately added and subtracted to each of the adjacent elements depending on the normal vector. Finally, the wave speeds computed when finding the fluxes are added to each adjacent cell. Now that the residual and wave speeds are generated on each cell, the timestep and state can be updated on each cell using the RK2 method outlined in 13.

Note that this code adheres to a linear time complexity constraint since only single, separate loops scale linearly with the mesh density.

### 3 Results

The results for the individual tasks requested by the project statement are presented and discussed.

#### 3.1 Task 1

The flux functions developed for the first and second order solvers are the Riemann Solvers Roe, Rusanov and HLLE. In order to test them a unit test is performed. In this test, an arbitrary input is proposed, with which the flux across the edge are calculated by hand. If the hand calculations match the flux functions outputs, it is a good indication that the flux implementation is correct. The same states and normal were used to test the three different fluxes:

$$\vec{U}_L = [0.9 \ 0.5 \ 0.1 \ 2]^T$$

$$\vec{U}_R = [1 \ 0.6 \ 0.15 \ 2.3]^T$$

$$\vec{n} = [0.5 \ \sqrt{3}/2]^T$$

By using the previous states and normal vector a hand calculation is developed in order to calculate the flux using the three different Riemann fluxes. There were obtained the following fluxes:

$$\vec{F}_{Roe} = [0.3238 \ 0.5576 \ 0.6904 \ 0.9916]^T$$

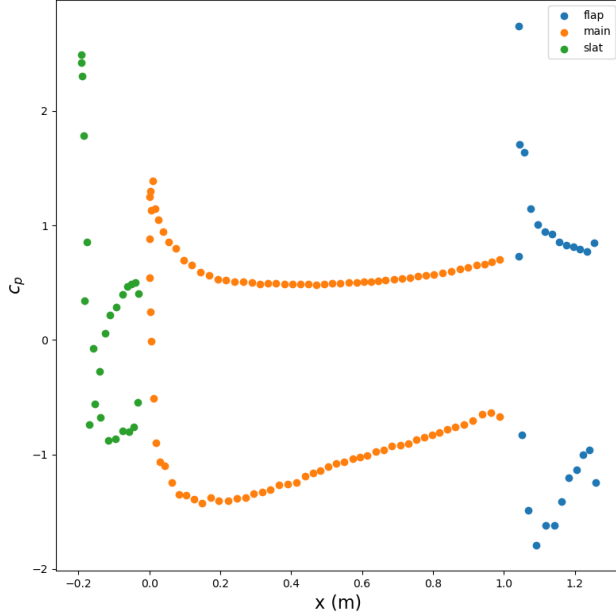
$$\vec{F}_{Rusanov} = [0.3074 \ 0.5431 \ 0.6996 \ 0.9610]^T$$

$$\vec{F}_{HLLE} = [0.3182 \ 0.5486 \ 0.6925 \ 0.9847]^T$$

By calling the developed functions for the three different Riemann fluxes in C++ using the proposed states and normal vector the results obtained are just the same as those obtained with by hand calculations. So the unit test performed was successfully passed, which is a good indication that the flux implementation is correct.

### 3.2 Task 2

Using the first order solver described in section 2.5, we solve for the coarse mesh using Rusanov flux. Figure 2 shows the Pressure-coefficient distribution over the airfoil for the subsonic flow case.



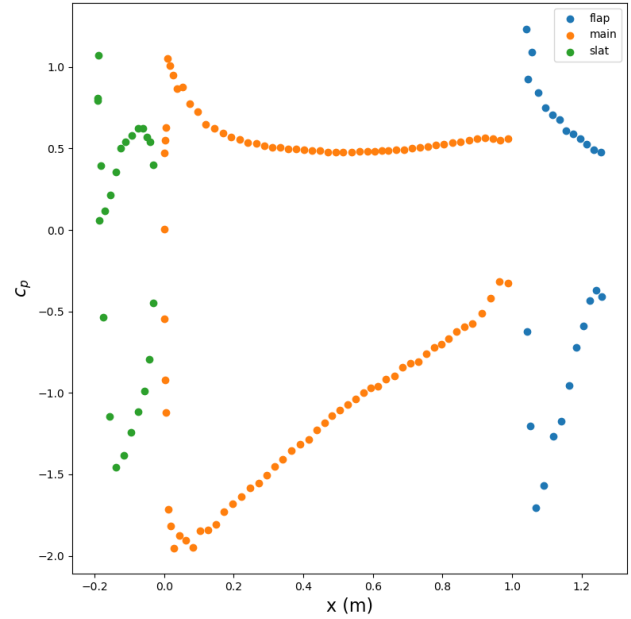
**Figure 2:** Pressure-coefficient distribution over the airfoil for first order simulation at free-stream Mach number 0.25 (subsonic) and angle of attack  $\alpha = 8^\circ$  using the coarse mesh

### 3.3 Task 3

Using the second order solver described in section 2.7, we solve for the coarse mesh using Rusanov flux and Barth-Jespersen (BJ) limiters. Figure 3 shows the Pressure-coefficient distribution over the airfoil for the subsonic flow case (same case as the first order solver for Task 2).

### 3.4 Task 4

In this section convergence studies on the subsonic flow condition are developed. As the scalar outputs in the studies we use the lift and drag coefficients. We have done three different studies. The first one is to analyze how the flux functions differ in dissipation. In order to do that we run the first order solver for the subsonic flow case using the



**Figure 3:** Pressure-coefficient distribution over the airfoil for second order simulation using BJ limiter at free-stream Mach number 0.25 (subsonic) and angle of attack  $\alpha = 8^\circ$  using the coarse mesh

3 fluxes developed. To compare them we use the number of iterations it took for each case to converge to  $1E-5$  L1 residual norm. Table 1 presents the iterations for each case.

**Table 1:** Iterations to converge to  $1E-5$  L1 residual norm.

| Flux    | # Iterations |
|---------|--------------|
| Roe     | 3687         |
| Rusanov | 1476         |
| HLLE    | 1786         |

It is possible to see in Table 1 that the most dissipative flux is Rusanov, as it takes the least amount of iterations to converge. Nevertheless, we can see that Roe flux is the least dissipative, but it is the most accurate of them all.

The second study is to analyze if the first and second order methods are actually first and second order accurate. To do so, we run the finest mesh

with the second order method solver for the subsonic case using no limiters and assume the drag and lift coefficients obtained in this simulation as the true values. Then, we run the first and second order solver with the medium size mesh for the same subsonic condition and computed the lift and drag coefficients. Table 2 presents the lift and drag coefficients for each case.

**Table 2: Lift and Drag coefficients**

| Case                | $c_d$    | $c_\ell$ |
|---------------------|----------|----------|
| Assumed true value  | 0.067215 | 3.325276 |
| First Order Solver  | 0.452352 | 2.312939 |
| Second Order Solver | 0.099452 | 3.042973 |

It is possible to see in Table 2 that the second order solver has one decimal place correct for the drag coefficient (the true and the second order both start with 0.0), while the first order solver has zero decimal places correct for the drag coefficient (the true and the first order both start with 0 but the first decimal is not correct). Also, considering the lift coefficient, the second order share the first number (the true and the second order both start with 3), while the first order does not share any number (the true value starts with 3 while the first order starts with 2). This show us that the first and second order solver are actually first and second order accurate, because the results obtained from the second order solver is one more decimal place accurate that the first order solver.

Finally, the last study is to analyze the impact of the limiters in the convergence rate. To do so, we run the second order solver for the subsonic flow case using the three possibilities for limiting (none, Barth-Jespersen (BJ) and the maximum-principle (MP) region limiter LCD). To compare them we use the converge L1 residual norm. Even though the goal is to reach a L1 residual norm of 1E-5, for the cases where we use limiters the L1 residual norm converge to a different value and then it hovers around that same value. We will use that numbers to compare the three cases. Table 3 presents the L1 residual norm to which each case

converges to (the non limiter case is stop at a L1 residual norm of 1E-5).

**Table 3: converged L1 residual norm**

| Limiter                | L1 residual norm |
|------------------------|------------------|
| None                   | 1E-5             |
| Barth-Jespersen (BJ)   | 0.02 (aprox)     |
| Maximum-Principle (MP) | 1.8 (aprox)      |

It is possible to see in Table 3 that the use of limiters highly affect the convergence of the solver. We can see that Barth-Jespersen (BJ) has better convergence than Maximum-Principle (MP), but even considering that it is still far from the value of 1E-5. So considering this we can say that the Maximum-Principle (MP) limiter is the one with the most impact on the convergence of the second order solver.

### 3.5 Task 5

Figure 4 and Figure 5 shows the computational mesh and Mach number contours for a first-order and second-order simulation of inviscid flow over the multi-element airfoil for the medium mesh case. Figure 6 and Figure 7 show the pressure-coefficient distribution over the airfoil for both the first order and second order using the medium mesh.

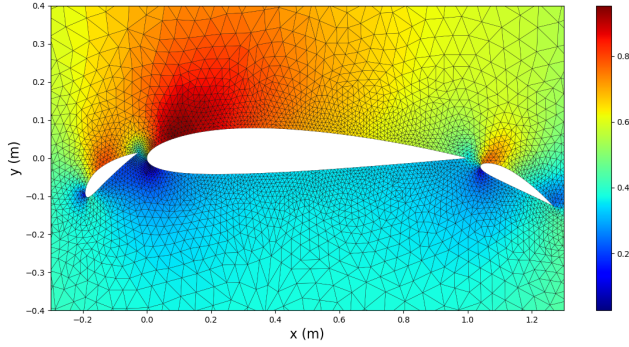


Figure 4: Mach contours for the multi-element airfoil at free-stream Mach number 0.5 (transonic) and angle of attack  $\alpha = 8^\circ$  using the first order solver with the medium size mesh

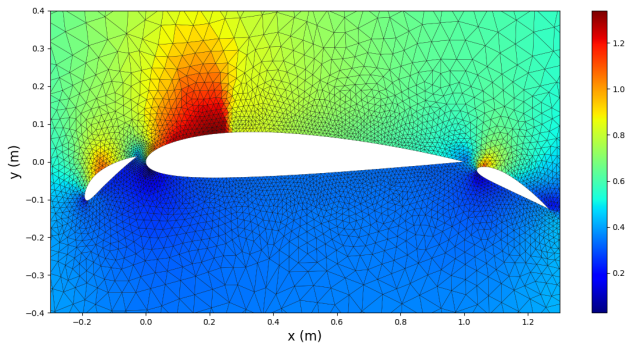


Figure 5: Mach contours for the multi-element airfoil at free-stream Mach number 0.5 (transonic) and angle of attack  $\alpha = 8^\circ$  using the second order solver with BJ limiter with the medium size mesh

It is possible to see that using the medium mesh, the results obtained with the first and second order are pretty different. We can observe that using a first order solver, the normal shock that is located above the main airfoil is not well obtained. Instead of having a discontinuity we are having a gradual change in the Mach number, which is not real. So just by looking at the medium mesh results we can see a key difference between first and second order.

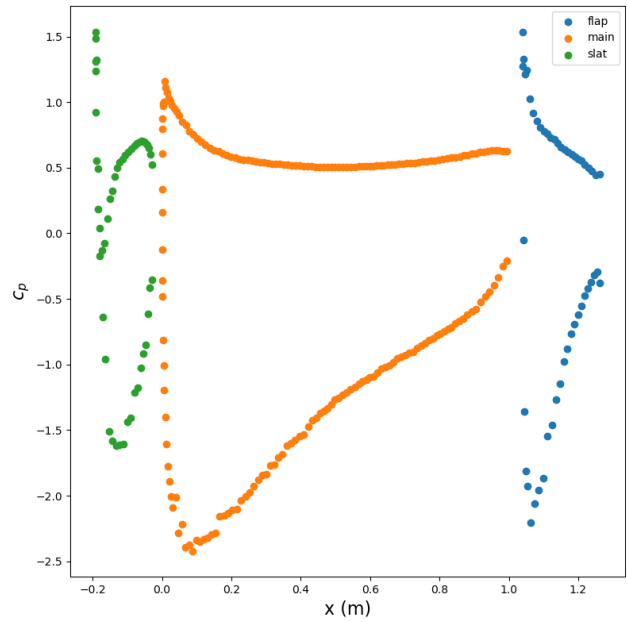


Figure 6: Pressure-coefficient distribution over the airfoil for first order simulations at free-stream Mach number 0.5 (transonic) and angle of attack  $\alpha = 8^\circ$  using the medium size mesh

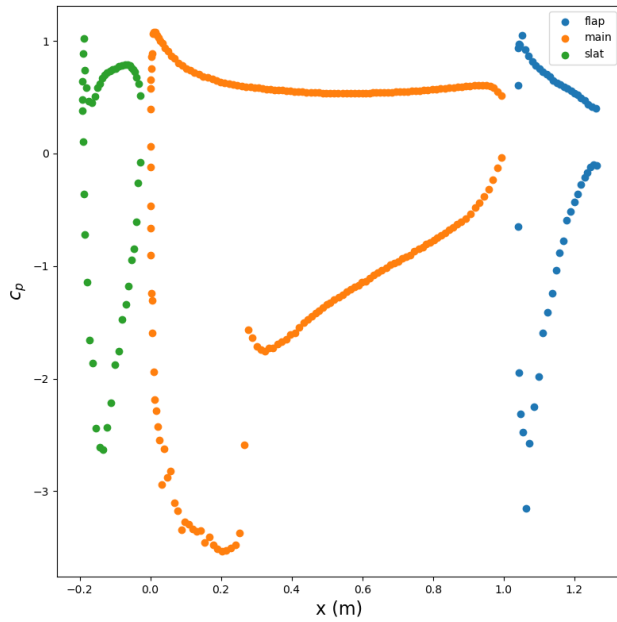


Figure 7: Pressure-coefficient distribution over the airfoil for second order (using BJ limiter) simulations at free-stream Mach number 0.5 (transonic) and angle of attack  $\alpha = 8^\circ$  using the medium size mesh

By looking at the Pressure-coefficient distribution we can see that the discontinuity due to the normal shock is very notorious for the second order case and it does not exist at all in the first order case. This shows us again that the first order solver is not able to catch the shock-wave, while the second order solves does.

Figure 8 and Figure 9 shows the computational mesh and Mach number contours for a first-order and second-order simulation of inviscid flow over the multi-element airfoil for the fine mesh case. Also, Figure 10 and Figure 11 shows the pressure-coefficient distribution over the airfoil for both the first order and second order using this finer mesh.

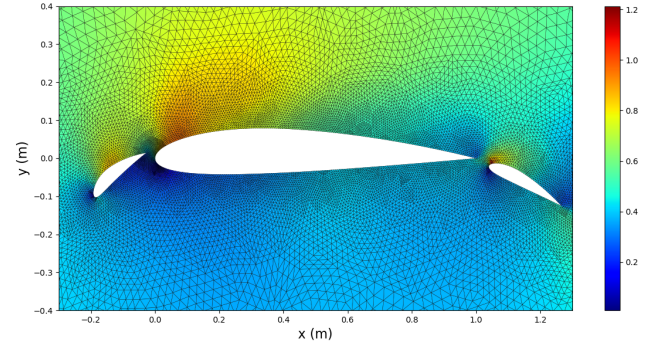


Figure 8: Mach contours for the multi-element airfoil at free-stream Mach number 0.5 (transonic) and angle of attack  $\alpha = 8^\circ$  using the first order solver with the fine mesh

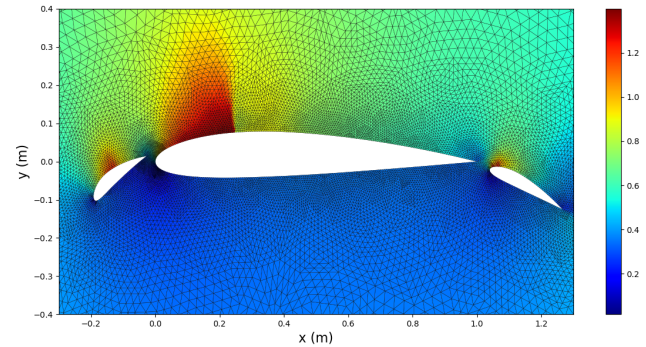
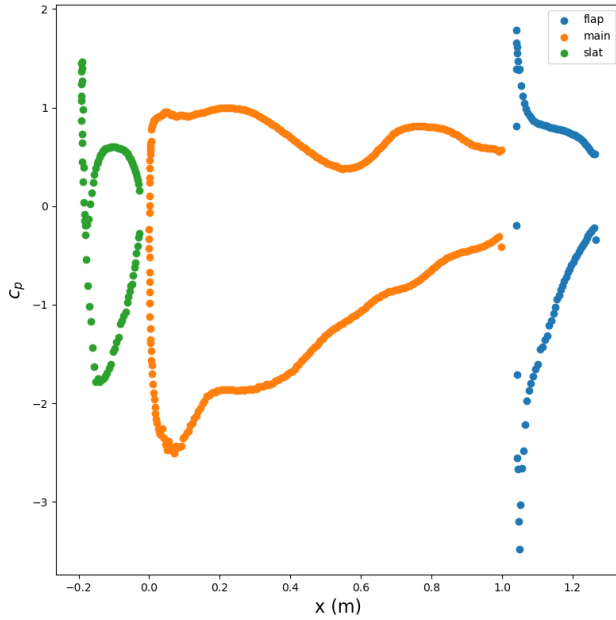
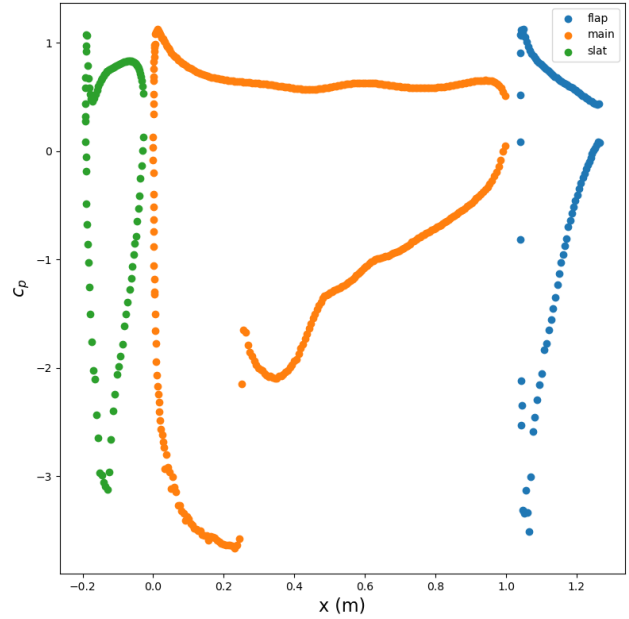


Figure 9: Mach contours for the multi-element airfoil at free-stream Mach number 0.5 (transonic) and angle of attack  $\alpha = 8^\circ$  using the second order solver with BJ limiter with the fine mesh

It is possible to see that using the finer mesh, the results obtained with the first and second order are still pretty different but the result that we got from the first order is closer to the discontinuity we are supposed to see. We still have a gradient instead of a discontinuity but it gets closer. Looking at the second order result we can see that the contours are better defined and that the normal shock is very well captured.



**Figure 10:** Pressure-coefficient distribution over the airfoil for first order simulations at free-stream Mach number 0.5 (transonic) and angle of attack  $\alpha = 8^\circ$  using the fine mesh



**Figure 11:** Pressure-coefficient distribution over the airfoil for second order (using BJ limiter) simulations at free-stream Mach number 0.5 (transonic) and angle of attack  $\alpha = 8^\circ$  using the fine mesh

Again, looking at the Pressure-coefficient distribution for the fine mesh we can see that the discontinuity due to the normal shock is still very notorious for the second order case and it does not exist in the first order case. Nevertheless, it is possible to see a drop in the pressure in the first order in the same place where the shock wave is. However, the drop in pressure is not as high as the one obtained with the second order solver, even with a coarser mesh.

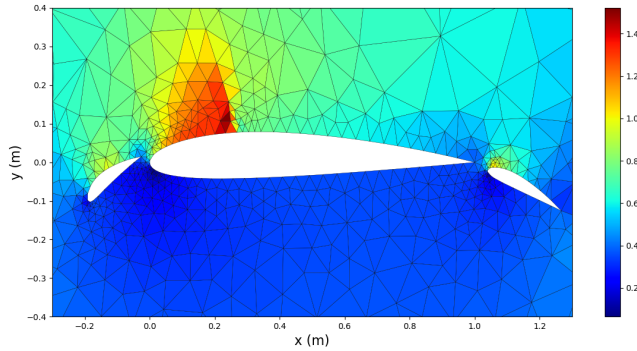
In order to be used in Task 6, the lift and drag coefficients were obtained for both the Fine mesh and the medium size mesh:

**Table 4:** Lift and Drag coefficients for adaptive run

| Case        | $c_d$    | $c_\ell$ |
|-------------|----------|----------|
| medium mesh | 0.218722 | 3.027350 |
| fine mesh   | 0.197466 | 3.172329 |

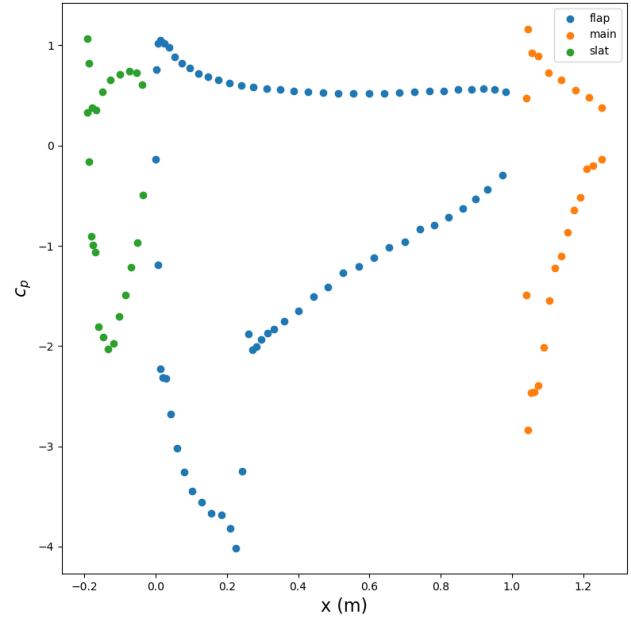
### 3.6 Task 6

Figure 12 below shows the Mach contours of the transonic flow condition on the coarse mesh after multiple mesh adaptations. When compared to Figure 9, which is the uniformly refined finest mesh, there are several similarities. While the fine mesh contours do seem more detailed, the range of the Mach number for both contours is similar as are the shapes of the contours.



**Figure 12:** Mach contours for the multi-element airfoil at free-stream Mach number 0.5 (transonic) and angle of attack  $\alpha = 8^\circ$  using the adaptive run

The  $C_p$  contours of the adapted mesh can be seen in Figure 13. Compared to the  $C_p$  of the fine mesh in Figure 11.



**Figure 13:** Pressure-coefficient distribution over the airfoil for second order (using BJ limiter) simulations at free-stream Mach number 0.5 (transonic) and angle of attack  $\alpha = 8^\circ$  using adaptive run

The results from the adapted run appear a little smoother. Considering that when doing the adaptation run starting from the coarse mesh we simulate using less cells, the adaptation took less time and still had accurate results.

**Table 5:** Lift and Drag coefficients for adaptive run

| $C_d$    | $C_l$    |
|----------|----------|
| 0.243307 | 2.962441 |

The lift and drag coefficients of the adaptive run can be seen in Table 5. While the results are close to that of the fine mesh in table 4, they are actually closer to the medium mesh results. However, the general trend of the finer mesh having a smaller  $C_d$  and larger  $C_l$  makes sense intuitively as the finer meshes will have better coverage of the viscous boundary layer.

## 4 Conclusions

The purpose of this project was to write a finite-volume solver to perform simulations of inviscid, compressible flow over the multi-element airfoil from the last project. Using the three meshes provided with the assignment, both first and second order solvers were implemented. In order to implement the solvers, flux functions were written, using Riemann solvers such as Roe, Rusanov, and HLLE. Special functions were written to take into account boundary conditions. The first order solver computed residuals on each element and integrated in time using Forward Euler and marched to steady state. In order to simulate the flow over the multi-element airfoil system using a second-order finite volume method, the state quantities near the cell faces were interpolated using gradients and limiters such as Barth-Jespersion and LCD. The second order solver took the converged first order solution as an initial guess and used RK2 for higher order time integration to march to steady state. From the relationship between L1-residual norm and number of iterations, it can be seen that the Rusanov flux function is the most dissipative, resulting in the highest convergence rate. Furthermore, the first order method was shown to be first order accurate, and the second order method was also shown to be second order accurate. The use of limiters also impacted the convergence rate, as neither the BJ nor MP limiters had L1 residual norms on the order of  $1E-5$ . Comparing the adaptive refinement results with the uniform refinement results, it is observed that the pressure distributions along the airfoil boundary become smoother during adaptive refinement because the jumps in the solution are targeted for smoothing.

## 5 Effort Breakdown

For this project, Taka and Nicolas worked on implementing the flux functions. After the completion of this implementation, Nicolas generated and ran verification tests to ensure the flux functions worked. Regarding the solvers, Devin worked on implementing the first order finite volume solver and also worked on both the forward Euler and

RK2 time integration methods. Jake implemented both the gradient and limiter computations as well as the second order solver. Nicolas provided various unit tests to ensure the functionality of the limiters. Building off of the solvers, Devin and Alex worked on the adaptive runs. Additionally, Taka and Alex worked on the post-processing together, specifically the plot generation and coefficient calculations. Finally, all group members contributed to the writing of the report. The breakdown of the effort in regards to various aspects of the project are shown in Table 6.

**Table 6: Effort percentages.**

|             | AF | DA | JY | NP | TS |
|-------------|----|----|----|----|----|
| development | 20 | 20 | 20 | 20 | 20 |
| coding      | 20 | 20 | 20 | 20 | 20 |
| managing    | 20 | 20 | 20 | 20 | 20 |
| report      | 20 | 20 | 20 | 20 | 20 |

## 6 References

### References

- [1] Krzysztof Fidkowski. Aero 623 course notes: Winter 2023. *University of Michigan, Ann Arbor, MI, 48109, USA*, 2023.
- [2] M. E. Hubbard. Multidimensional slope limiters for muscl-type finite volume schemes on unstructured grids. *Journal of Computational Physics*, vol. 155, 1999.